



S.E.P. TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO de Tuxtepec

PROGRAMACIÓN NATIVA PARA PLATAFORMAS MOVILES

“REPORTE DEL CURSO DE CONCEPTOS BASICOS DE FLUTTER”

PRESENTA:

ITURBIDE MARIANO VICTOR MANUEL

N. DE CONTROL:

22350400

SEMESTRE: 8

GRUPO: A

DOCENTE:

JULIO AGUILAR CARMONA

ESPECIALIDAD:

INGENIERIA INFORMÁTICA

FECHA DE ENTREGA:

12/JUNIO/2026



INTRODUCCION

Durante el desarrollo del curso Introduction to Flutter, adquirí conocimientos fundamentales sobre el desarrollo de aplicaciones móviles multiplataforma utilizando Flutter y el lenguaje de programación Dart. A lo largo del curso aprendí desde la configuración inicial del entorno de desarrollo hasta la creación de una aplicación funcional que consume información desde una API externa.

Este curso me permitió comprender la estructura básica de una aplicación Flutter, el funcionamiento de los widgets, el manejo de interfaces gráficas y la comunicación con servicios web, fortaleciendo mis habilidades en el desarrollo móvil.

TEMAS VISTOS

1. Configuración del Entorno de Desarrollo

Aprendí a instalar Flutter y configurar correctamente el entorno de trabajo para comenzar a desarrollar aplicaciones móviles. También comprendí la importancia del SDK de Flutter y de contar con las herramientas necesarias para ejecutar y probar aplicaciones.

Ejemplo realizado

Mi primera aplicación consistió en mostrar un texto dentro de una ventana básica:

```
void main() {
  runApp(
    MaterialApp(
      home: Scaffold(
        appBar: AppBar(
          title: Text('Mi Primera App'),
        ),
        body: Center(
          child: Text('Hola Flutter'),
        ),
      ),
    ),
  );
}
```

Con este ejemplo entendí cómo se estructura una aplicación Flutter y cómo se organizan sus componentes principales.

2. Introducción a Dart y Programación Orientada a Objetos

Antes de profundizar en Flutter, aprendí conceptos básicos de Dart, incluyendo variables, funciones, clases y objetos. Comprendí cómo la programación orientada a objetos facilita la organización del código y mejora su reutilización.

Ejemplo realizado

```
class Persona {
  String nombre;

  Persona(this.nombre);
}

void main() {
  Persona persona = Persona("Juan");
  print(persona.nombre);
}
```

Este ejercicio me ayudó a entender cómo crear objetos y almacenar información dentro de clases.

3. Uso de Widgets en Flutter

Aprendí que los widgets son los componentes fundamentales de Flutter y que toda la interfaz gráfica está construida mediante ellos. Trabajé con widgets como:

- Text
- Container
- Row
- Column
- ListView
- ListTile
- AppBar
- Icon
- SnackBar
- SingleChildScrollView

Comprendí cómo combinar widgets para construir interfaces organizadas y visualmente atractivas.

Ejemplo realizado

```
Column(  
  children: [  
    Text("Nombre"),  
    Text("Edad"),  
    Text("Carrera"),  
  ],  
)
```

Con este ejercicio aprendí a organizar elementos de forma vertical.

4. Widgets Stateless y Stateful

Uno de los temas más importantes fue comprender la diferencia entre widgets estáticos y dinámicos.

Comprendí que:

- Un StatelessWidget no cambia su contenido una vez creado.
- Un StatefulWidget puede actualizarse durante la ejecución mediante el método `setState()`.

Ejemplo realizado

```
class Contador extends StatefulWidget {
  @override
  _ContadorState createState() => _ContadorState();
}

class _ContadorState extends State<Contador> {
  int contador = 0;

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        Text("$contador"),
        ElevatedButton(
          onPressed: () {
            setState(() {
              contador++;
            });
          },
          child: Text("Incrementar"),
        ),
      ],
    );
  }
}
```

Este ejemplo me permitió entender cómo actualizar la interfaz cuando cambian los datos.

5. Creación y Validación de Formularios

Aprendí a crear formularios para capturar información del usuario y validar los datos ingresados antes de procesarlos.

Comprendí la importancia de validar información para evitar errores y mejorar la experiencia del usuario.

Ejemplo realizado

```
TextFormField(  
  decoration: InputDecoration(  
    labelText: "Correo",  
  ),  
  validator: (value) {  
    if (value!.isEmpty) {  
      return "Campo obligatorio";  
    }  
    return null;  
  },  
)
```

Con este ejercicio aprendí a verificar que los campos sean llenados correctamente.

6. Navegación entre Pantallas

Aprendí a moverme entre diferentes pantallas dentro de una aplicación utilizando el sistema de navegación de Flutter.

También comprendí cómo enviar y recuperar información entre vistas.

Ejemplo realizado

```
Navigator.push(  
  context,  
  MaterialPageRoute(  
    builder: (context) => SegundaPantalla(),  
  ),  
);
```

Para regresar a la pantalla anterior:

```
Navigator.pop(context);
```

Este tema fue fundamental para comprender cómo se estructuran aplicaciones con múltiples interfaces.

7. Manejo de Datos JSON

Aprendí a trabajar con archivos JSON, interpretar su contenido y convertirlo en objetos utilizables dentro de la aplicación.

También comprendí cómo los servicios web utilizan este formato para intercambiar información.

Ejemplo realizado

```
import 'dart:convert';

void main() {
  String jsonData = '''
  {
    "nombre": "Juan",
    "edad": 21
  }
  ''';

  var datos = jsonDecode(jsonData);

  print(datos["nombre"]);
}
```

Con este ejercicio aprendí a convertir datos JSON en estructuras que pueden ser utilizadas por Flutter.

8. Consumo de APIs mediante HTTP

Aprendí a conectar aplicaciones Flutter con servicios externos mediante peticiones HTTP.

Comprendí cómo solicitar información a una API y utilizar las respuestas para mostrar contenido dinámico dentro de la aplicación.

Ejemplo realizado

```
final response = await http.get(  
  Uri.parse("https://api.example.com/data"),  
);
```

Este conocimiento me permitió entender cómo las aplicaciones modernas obtienen información desde Internet.

9. Programación Asíncrona con FutureBuilder

Aprendí a manejar procesos asíncronos mediante FutureBuilder, lo que permite mostrar información obtenida desde servidores sin bloquear la interfaz de usuario.

Ejemplo realizado

```
FutureBuilder(  
  future:                                obtenerDatos(),  
  builder: (context, snapshot) {  
    if (snapshot.hasData) {  
      return Text(snapshot.data.toString());  
    }  
  
    return CircularProgressIndicator();  
  },  
)
```

Con este ejercicio comprendí cómo mostrar indicadores de carga y actualizar la interfaz cuando llegan los datos.

10. Desarrollo del Proyecto Final: Earthquake App

Durante el proyecto final integré todos los conocimientos adquiridos en el curso.

La aplicación desarrollada consulta información sobre terremotos desde una API externa y muestra los resultados al usuario.

En este proyecto puse en práctica:

- Diseño de interfaces.
- Navegación.
- Consumo de APIs.
- Procesamiento de JSON.
- FutureBuilder.
- Manejo de datos remotos.

Ejemplo realizado

```
Future<List> obtenerTerremotos() async {  
  var response = await http.get(  
    Uri.parse(apiUrl),  
  );  
  
  return jsonDecode(response.body);  
}
```

Este proyecto me permitió comprender el flujo completo de desarrollo de una aplicación Flutter funcional.